Appendix C: IEEE 830 Template

Software Requirements Specification

for

«Project Name»

Version «# matching the last line of the versions table»

Prepared by «author»

«organisation»

«date created»

«Page Break»

Table of Contents

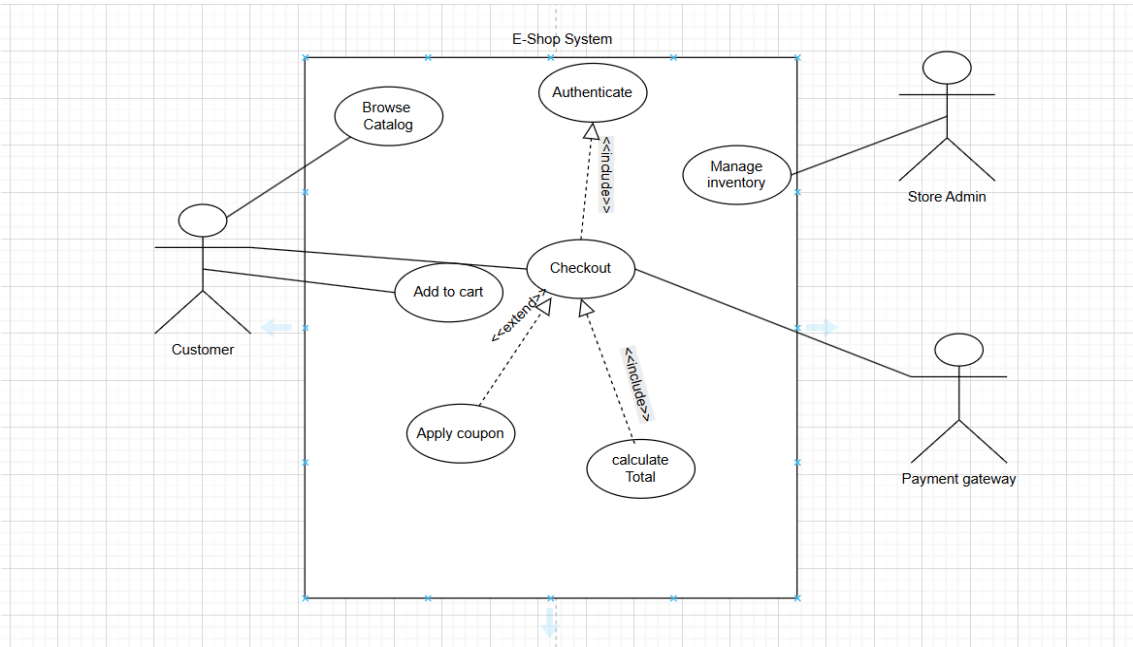
«Table of Contents»

Revision History

Name	Date	Reason for Changes	Version
Srdjan Saranovic	04.03.2026	«"Initial version" or actual reason»	V1
Srdjan Saranovic	24.03.2026.	Added UML Use Case Diagram and completed section 1	V1.1

«Page break»

1. Introduction



1.1. Purpose

This Software Requirements Specification (SRS) defines the requirements for the E-shop System v1.0. It describes both functional and non-functional requirements for customer and admin interactions. The system enables users to browse products, add items to a shopping cart, and complete purchases through a checkout process, as illustrated in the use case diagram.

1.2. Document Conventions

This document follows standard SRS structure and uses consistent formatting:

- Functional requirements are labeled as **REQ-x.x**
- Non-functional requirements are labeled as **NF-x.x**
- Priorities are indicated (e.g., High, Medium)
- Use case names (e.g., *Checkout*, *Add to Cart*) match those shown in the diagram
  All requirements are written clearly and each has its own priority where applicable.

1.3. Intended Audience and Reading Suggestions

This document is intended for:

- **Developers** – to understand system functionality and implementation requirements
- **Testers** – to validate system behavior against requirements
- **Project managers** – to track scope and priorities
- **Designers** – to understand system interactions
- **Stakeholders** – to get an overview of system capabilities

It is recommended to first read the overview (Section 1), then the functional requirements (Section 5), followed by non-functional requirements (Section 6). The use case diagram should be reviewed alongside these sections for better understanding.

1.4. Product Scope

The software is an online retail platform that allows users to browse products, manage a shopping cart, and perform secure checkouts. The goal is to increase sales efficiency and provide a 24/7 shopping experience.

1.5. References

- IEEE 830 Software Requirements Specification Standard
- Course materials and assignment description
- UML Use Case Diagram guidelines
- BCrypt password hashing documentation
- SSL/TLS security standards documentation

2. Overall Description

2.1. Product Perspective

This is a **new, self-contained product**.

2.2. Product Features

- Product Catalog Browsing.
- Shopping Cart Management.
- Secure Payment Processing.
- Order Tracking.

2.3. User Classes and Characteristics

«Identify the various user classes (intended target for the product) that you anticipate will use this product. User classes may be differentiated based on frequency of use, subset of product functions used, technical expertise, security or privilege levels, educational level, or experience. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes. Distinguish the most important user classes for this product from those who are less important to satisfy.»

2.4. Operating Environment

«Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.»

2.5. Design and Implementation Constraints

«Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).»

2.6. User Documentation

«List the user documentation components (such as user manuals, on-line help, and tutorials) that will be delivered along with the software. Identify any known user documentation delivery formats or standards.»

2.7. Assumptions and Dependencies

«List any assumed factors (as opposed to known facts) that could affect the requirements stated in the specification. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).»

3. System Features

«The purpose here is to give a brief introduction to the system features, so that §4 has something to reference. Then, the full explanation of the system features is given in §5, likely referencing §4. The numbering in §3 and §5 should match.»

3.1. «Feature Name»

«Brief description of the feature, generally 1-3 sentences»

3.«N». «Additional feature blocks as needed»

4. External Interface Requirements

4.1. User Interfaces Overview

«Describe the high level functionality of the system from the user's perspective. Explain how the user should be able to use your system to complete all the expected features and the feedback information that will be displayed for the user (for each screen the user will see). For example, specify whether it is GUI or text based interface and how at high level it would work. Discuss at high level how user will interact with the game such as clicking buttons, typing in some selection character, etc.»

«Describe the minimum requirements for the interface. This may include some sample screen images whether for text or GUI approach, any GUI standards or product family style guides that are to be followed, screen layout constraints, for GUI standard buttons and functions (e.g., help) that must appear on every screen, keyboard shortcuts, error message display standards, and so on»

4.2. Hardware Interfaces

«Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.»

4.3. Software Interfaces

«Describe the connections between this product and other specific software components (name and version), including databases, interpreters, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.»

5. System Features/Modules

«This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product.»

5.1. Shopping Cart Module

5.1.1 Description and Priority

Allows users to add, remove, and update quantities of items. **Priority: High**.

5.1.2 Stimulus/Response Sequences

Condition: User clicks "Add to Cart".

Response: System updates the cart count and saves the item session.

5.1.3 Functional Requirements

REQ-1.1: Upon clicking "Add to Cart", the system shall verify if the item is in stock.

REQ-1.2: The system shall calculate the total price including taxes at the checkout stage.

6. Nonfunctional Requirements

6.1. Performance

NF-1.1: The search results shall be displayed within 1.5 seconds.

6.2. Security

NF-2.1: The system shall use SSL for all payment transactions.
NF-2.2: User passwords must be hashed using BCrypt before being saved in the database.